

Plano de Ação Autônomo: TASK-AUT-001 (TOOL-ART-001)

1. Rastreamento de Ciclo de Governança

Este artefato documenta a primeira iniciativa de criação autônoma de código sob a égide do protocolo DTGE.

- **id_plano:** PLANO-TASK-AUT-001
- **tipo_plano:** AUTONOMOUS_CREATION
- **profundidade_ciclo:** 1
- **Timestamp (UTC):** 2024-05-22T16:45:00Z
- **Referência Técnica:** Derivado da diretiva **TOOL-ART-001** .

2. Objetivo e Escopo Técnico

- **Objetivo Central:** Desenvolver um script Python (.py) especializado na validação de integridade e conformidade de estruturas JSON_COMPACTO. O objetivo é mitigar o consumo excessivo de tokens e garantir que os payloads de comunicação entre agentes sigam o padrão de separação otimizada.
- **Dentro do Escopo:**
 - Implementação da lógica de validação de schema JSON utilizando a constante JSON_COMPACTO importada de dreamteam_core.py.
 - Verificação rigorosa da ausência de espaços em branco (whitespace) entre chaves e valores, garantindo o padrão separators=(" ", ":").
 - Integração com o fluxo de registro para submissão de metadados à cadeia canônica.
- **Fora do Escopo:**
 - Alteração de contratos já "Congelados" (Status: CONGELADO) em _BASE/CANONIZACOES/.
 - Modificação do código-fonte central do agente ferramenta_registrador_standalone.py.

3. Checagem Anti-Drift (RC-ASSIST-001)

Em conformidade com as obrigações do Arquiteto DTGE:

- **Auditoria de Base:** Foi executada a lógica de _consultar_canonizacoes (conforme definido em ferramenta_arquiteto_standalone.py) no diretório _BASE/CANONIZACOES/.
- **Resultado:** Confirmou-se a inexistência de um "Módulo de Validação de Schema" previamente canonizado.
- **Mandato de Extensão:** Fica estabelecido que qualquer nova utilidade de manipulação de JSON não deve ser duplicada; o script deve ser proposto como uma extensão funcional de dreamteam_core.py. É proibido o uso de "magic strings" para definições de formato que já constam no núcleo do sistema.

4. Requisitos de Implementação e Segurança

O desenvolvimento do artefato deve seguir as seguintes "Regras Duras":

- **Padrão de Codificação:** Codificação UTF-8 (# -- coding: utf-8 --) e adesão total às convenções de nomenclatura de dreamteam_core.py.

- **Blindagem e Integridade:** O script deve ser compatível com a função `validar_integridade` do módulo `blindagem_dtge.py`. Após a criação, o script deve invocar `registrar_manifesto(nome_script, autorizado_por)` para assegurar que seu hash entre na base de confiança.
- **Validação Técnica:** A lógica deve validar não apenas a estrutura (schema), mas a compactação física do arquivo, rejeitando qualquer JSON que possua indentação ou espaços decorativos.

5. Critérios de Aceite (Definition of Done - DoD)

A tarefa só será considerada finalizada mediante o cumprimento integral destes itens:

- **Persistência de Código:** Arquivo `.py` funcional e sanitizado disponível no sandbox.
- **Bateria de Testes (Dual):**
- **Teste Positivo:** Validação bem-sucedida de um payload em formato `JSON_COMPACTO` legítimo.
- **Teste Negativo:** Rejeição explícita de um payload JSON indentado ou com espaços.
- **Registro na Cadeia:** Ingestão do `pedido_de_registro` na `CADEIA.jsonl`. O sucesso é condicionado à obtenção de um número de sequência (**Seq**) válido gerado pelo Registrador.
- **Evidência de Integridade:** Geração do hash SHA256 do script via `TOOL-EVD-002`, armazenado obrigatoriamente em um arquivo `.sha256.txt`.

6. Mecanismo de Autorização e Execução

- **Protocolo:** A execução deste plano segue o `PROTOCOLO_DREAMTEAM_CLAUDE_v1.md`.
- **Token de Autoridade:** A execução é permitida exclusivamente via **Token de Autoridade Delegada** (`TOOL-EVT-005`).
- **Restrição:** É terminantemente proibida a execução via tokens de avanço cognitivo (ex: `SEGUIR`).
- **Rastreabilidade:** O `event_id` do token de autorização deve ser capturado, logado na evidência de execução e incluído nos metadados do pedido de registro enviado ao Registrador.

7. Estratégia de Rollback (`TOOL-EVT-003`)

Em caso de falha crítica nos testes ou erro de sintaxe:

- **Procedimento:** Exclusão imediata do arquivo gerado no sandbox.
- **Tamper-Evidence:** A falha deve ser sinalizada no `INDICE.jsonl`.
- **Encadeamento:** O evento de rollback deve utilizar o campo `prev_ref` para apontar para o último estado íntegro conhecido, garantindo que a `CADEIA.jsonl` permaneça auditável e livre de referências a artefatos corrompidos.

8. Perguntas de Ancoragem (`TOOL-COG-007`)

- **O que valida o sucesso?** A validação técnica do script em ambiente de sandbox, o registro de seu hash via Blindagem e a confirmação de ingestão na cadeia imutável.
- **O que esta tarefa não é?** Não é uma reestruturação do `ferramenta_registrador_standalone.py` nem uma mudança nos contratos canonizados de governança.

- **Qual a evidência mínima de conclusão?** Log de execução contendo o event_id da autorização, o arquivo .sha256.txt e o número de sequência (**Seq**) correspondente na CADEIA.jsonl.
- **O que é reversível?** A criação do arquivo físico do script e as entradas temporárias no sandbox, antes da canonização final.